MINISTRY OF EDUCATION & TRAINING
HOCHIMINH CITY UNIVERSITY OF TECHNOLOGY

GRADUATION PROJECT

PROGRESSIVE SKILL GAP ANALYSIS AND COURSE RECOMMENDATION VIA LLM EXTRACTION AND HIERARCHICAL TREE SEARCH

Subject: **Information Technology**

Specialty: **Information Technology**

Supervisor (Lecturer): Dr. Lê Thị Ngọc Thơ

Student's name: Nguyễn Tùng Dương

Student's ID: 2280600389

Class: 22DTHQA1

Ho Chi Minh City, June 2026

TABLE OF CONTENTS

LIST OF FIGURES v

LIST OF TABLES vi

ABSTRACT vii

CHAPTER 1 INTRODUCTION 1

1.1 Motivation 1

1.2 Problem Statement 1

1.3 Research Objectives 2

1.4 Scope of the Study 2

1.4.1 System Inputs 2

1.4.2 System Outputs 3

1.4.3 Research Delimitations 4

1.5 Thesis Structure 5

CHAPTER 2 PRELIMINARIES AND RELATED WORKS 6

2.1 Preliminaries 6

2.1.1 Large Language Models (LLMs) and Prompt Engineering 6

2.1.2 Tree-based Search and Depth-First Traversal 6

2.1.3 In-memory Caching and Short-Circuit Evaluation 7

2.2 Related Works 7

2.2.1 Traditional Job Matching and Keyword-based NLP 7

2.2.2 AI-driven Recommendation in EdTech and LLMTreeRec 8

2.2.3 Limitations of Prior Work and the Position of This Thesis 8

CHAPTER 3 A HYBRID FRAMEWORK FOR SKILL GAP ANALYSIS AND COURSE RECOMMENDATION 10
3.1 System Overview 10
3.1.1 Offline Phase: Hierarchical Course Tree Construction 11
3.1.2 Online Phase: The Recommendation Pipeline 11
3.2 Offline Phase: Building the Hierarchical Course Tree 11
3.2.1 Data Pipeline 12
3.2.2 Hierarchical Tree Construction 12
3.3 Skill & Level Extraction 13
3.3.1 Pre-processing 14
3.3.2 Prompt Design and LLM Extraction 14
3.3.2.1 Zero-Tool & Blacklist Rule: 14
3.3.2.2 Propositional Logic Extraction: 14
3.3.2.3 Level Quantification: 15
3.3.2.4 JSON Slicing Mechanism: 15
3.3.3 Global Alias Mapping 15
3.4 Computing skill gap 16
3.4.1 Dynamic Alias Resolution 16
3.4.2 Disjunctive Skill Groups 16
3.4.3 Progressive Level Array Calculation 17
3.5 Course Retrieval by Deterministic Tree Search 18
3.5.1 Problem and Approach 18
3.5.2 Depth-First Traversal 19
3.5.3 Deterministic Skill-set Pruning 19
3.5.4 Retrieval Stage Results 20

3.6 Deterministic Ranking and the Evaluator 20
3.6.1 Combo Courses via Set Cover 20
3.6.2 Deterministic Relevance Ranking 21
3.6.3 The Evaluator E(c) 21
3.6.4 Quality Metrics: Precision@K and Coverage 22
CHAPTER 4 EXPERIMENTS AND EVALUATIONS 23
4.1 Experimental Setup 23
4.1.1 Datasets 23
4.1.2 Implementation Environment 24
4.1.3 Evaluation Protocol and Metrics 24
4.2 Skill Extraction Accuracy (P1) 25
4.3 Token Consumption Analysis (P2) 27
4.4 Recommendation Quality and Guardrail Ablation (P3) 28
4.5 Retrieval (Pruning) Strategy Ablation (P4) 29
4.6 Ranking Strategy Ablation (P5) 30
4.7 Discussion 32
4.7.1 Synthesis: Where the LLM Belongs 32
4.7.2 Threats to Validity 32
4.7.3 Observed Limitations 33
CHAPTER 5 CONCLUSION & FUTURE WORK 34
5.1 Summary of Contributions 34
5.2 System Limitations 35
5.2.1 Single-label Taxonomy and Tag-based Matching 35
5.2.2 Static Data Obsolescence 35

5.2.3 Cloud API Dependency 35
5.2.4 Extreme Cold-Start Threshold: 36
5.2.5 Evaluation Scope and Ground Truth 36
5.3 Future Work 36
5.3.1 From Tree to Knowledge Graph 36
5.3.2 Automated Catalogue Maintenance 36
5.3.3 Local LLM Deployment 37
5.3.4 Dynamic Alias Discovery 37
REFERENCES 38

LIST OF FIGURES

Figure 3-1 Build Hierarchical Course Tree. 10
Figure 3-2 Overall Flowchart. 10
Figure 3-3 Course tree structure 13
Figure 4-1 Precision, Recall, and F1 of keyword baseline versus LLM extraction. 26
Figure 4-2 Token consumption per query on a logarithmic scale 27
Figure 4-3 Retrieval recall for the three pruning strategies. 29
Figure 4-4 Latency for the three pruning strategies 30
Figure 4-5 Ranking precision and reproducibility (Jaccard × 100) per strategy. 31

LIST OF TABLES

Table 1-1 Extract Skill and Level of CV (Su) 3
Table 1-2 Extract Skill and Level of JD (Sj) 3
Table 1-3 Compute skill gap. 4
Table 3-1 Mapping from job-description wording to proficiency level. 15
Table 4-1 Summary of datasets used in the evaluation. 24
Table 4-2 The five evaluation experiments and their purpose. 25
Table 4-3 Skill extraction: raw counts and Precision/Recall/F1 over 36 JDs (131 gold skills). 25
Table 4-4 Average tokens sent to the LLM per query (mean over 100 cases). 27
Table 4-5 Recommendation quality with and without the guardrail E(c) (36 pairs). 28
Table 4-6 Pruning strategy ablation (averages over 36 pairs). 29
Table 4-7 Ranking strategy ablation: precision, reproducibility, and LLM cost. 31

ABSTRACT

In the Vietnamese labour market, and in the IT sector in particular, many candidates are unsure which skills they are missing or whether their current skills match what employers ask for, which leaves them hesitant when they apply. They also do not know which courses to take to close those gaps. Traditional skill-matching cannot capture complex requirements: a posting that asks for "C#, Java, or Python" is read as three mandatory skills rather than one optional choice, and equivalent proficiency levels are missed. Using a Large Language Model (LLM) to run the whole process end to end is not a safe alternative either, because its output is hard to reproduce and to verify. We therefore propose a hybrid method. An LLM is used only to extract skills and proficiency levels from CVs and job descriptions; a mathematical module then computes the skill gap; the candidate courses are evaluated and ranked; and finally a set of suitable courses is recommended to the user.

The results are as follows. For extraction, the system reached an F1-score of 96.7%, against only 84.4% for the traditional keyword method, and it was able to handle "OR" requirements and to read proficiency levels. Removing the LLM from the later steps, namely gap computation, search, and ranking, did not reduce quality. Deterministic pruning reached 99.2% recall, compared with 63.3% for an LLM-routed baseline, while running about twenty-five times faster and making no per-query LLM calls. Deterministic ranking matched the precision of LLM ranking (75.4% versus 73.8%) and returned the same result on every run, and the end-to-end recommendations reached 74.8% precision against human labels. The volume of data sent to the LLM was reduced by 99.8%, a saving that comes from the deterministic filter rather than from the tree structure.

The resulting system is a deterministic variant of LLM-guided tree retrieval. Apart from the extraction step it is fully reproducible, requires no further LLM calls, and cannot hallucinate at its deterministic stages, since no model runs there. It provides a practical framework for bridging the gap between recruitment demands and educational supply.

Keywords: skill gap analysis, course recommendation, large language models, deterministic retrieval, prompt engineering, reproducibility

CHAPTER 1 INTRODUCTION

1.1 Motivation

The IT industry changes quickly, and the skills employers ask for often differ from what students are taught at university. Online recruitment sites such as ITviec and VietnamWorks are widely used, yet many job seekers, especially recent graduates and engineers with only one or two years of experience, cannot tell whether their current skills match the requirements of a posting they are interested in.

The deeper issue is one of awareness. A candidate often cannot name the exact skills they are missing, cannot judge how far their current level sits from what a role expects, and has no clear place to start. The supply side does not make this easier. Online courses now number in the tens of thousands, with new ones appearing every day, so choosing the few that genuinely advance a career goal becomes guesswork. A system that reads a candidate profile, measures the gap against a specific job, and recommends courses aimed at that gap would meet a real and common need.

1.2 Problem Statement

Connecting job requirements (JDs), candidate profiles (CVs), and educational supply (courses) automatically raises two difficulties that shape the rest of this work.

- The first is semantic blindness. Conventional applicant-tracking systems rely on keyword techniques such as regular expressions, TF-IDF, or Jaccard similarity. Because they treat text as a set of isolated tokens, they cannot see that “C# or Java” is an alternative rather than two mandatory skills, and they cannot tell a beginner’s “exposure to Python” from a senior engineer’s “Python architecture.” The result is coarse, often inaccurate matching.
- The second is the unreliability of an end-to-end LLM. A large language model reads such text well, but used on its own as a recommender it tends to invent skills, misread levels, or suggest courses that do not fit, and its output changes from one run to the next. For a tool meant to guide a person’s learning, that inconsistency is a serious drawback.

1.3 Research Objectives

This thesis designs and builds a hybrid system for skill-gap analysis and course recommendation. The guiding principle is to rely on the language model only where natural-language understanding is unavoidable, and to keep every other stage deterministic. The specific objectives are as follows.

- To build an extraction pipeline that uses a prompt-constrained LLM (Qwen3-Coder-480B) to convert unstructured CVs and JDs into a structured set of skills and proficiency levels, with rules that limit fabricated or out-of-scope output.
- To formulate a mathematical skill-gap calculator that measures, for each required skill, both whether it is missing and how many proficiency levels separate the candidate from the target.
- To design a retrieval method that organises the catalogue as a hierarchical tree and finds relevant courses by deterministic skill-set matching, rather than by querying the LLM at every branch.
- To add a deterministic ranking step and an evaluator E(c) that audit the recommended courses, so that the final learning path is more reliable and fully reproducible.
- To evaluate the system empirically, and in particular to measure where the LLM genuinely adds value and where deterministic computation is sufficient.

1.4 Scope of the Study

1.4.1 System Inputs

The system works with three inputs. From a CV it derives the candidate's existing skills, each paired with a self-assessed level. From a JD it derives the skills a role requires together with the expected level for each. The third input is the course catalogue, a hierarchical collection of courses where each course is described by the skills it teaches and the level at which it teaches them. As a running example, consider the following CV and JD for a Backend Java position.

- From the CVs, we extract the user profile.

  Represented as a set of existing skills $S_u = \{s_1, s_2, \ldots, s_m\}$, In which each skill is associated with a self-assessed proficiency level:

  L ∈ {1: Basic, 2: Intermediate, 3: Advanced}.

Example: **CV:** “Developer with one year of experience. Worked with Spring Boot at a basic level. Proficient in MySQL. Familiar with Git.”. From the above CV, we can deduce the following:

Table 1-1 Extract Skill and Level of CV (S_u)

Skill	Level
Spring Boot	1
MySQL	3
Git	2

- From the employer's JD, we extract the target skills.

  Represented as a set of required skills $S_j = \{s_1, s_2, \ldots, s_n\}$

  In which each skill is associated with a self-assessed proficiency level:

  L ∈ {1: Basic, 2: Intermediate, 3: Advanced} and optional types: Extract, Any.

  Example: **JD:** “Backend Java Developer. Proficient in Spring Boot. Experience with Microservices. Good command of PostgreSQL or MySQL. Docker is a plus.” From the above JD, we can extract:

Table 1-2 Extract Skill and Level of JD (S_j)

Skill	Level	Type
Spring Boot	3	Extract
Microservices	2	Extract
PostgreSQL | MySQL	3	Any
Docker	1	Extract

- Course Catalog (Educational Supply):

  A static, hierarchical database of courses $C = \{c_1, c_2, \ldots, c_k\}$. Each course c_i provides a specific set of target skills S_{ci} and operates at a defined pedagogical level.

1.4.2 System Outputs

From these inputs the deterministic engine produces two results.

- Quantified Skill Gap (Gap). For each skill s, let $l_u(s)$ be the candidate’s level (0 if the skill is absent) and l_j(s) the level the job requires. A required skill

belongs to the gap whenever the candidate's level is below the required level, which covers both a missing skill ($l_u = 0$) and an insufficient level ($0 < l_u < l_j$). Each gap skill carries the ordered list of levels still to be acquired:

$$L_s = \langle max(1,\ l_u(s) + 1),\ \ldots,\ l_i(s) \rangle$$

$$Gap = \{(s, L_s) : s \in S_j,\ l_u(s) < \ell_j(s)\}$$

The naive set difference $S_j - S_u$ is only the special case that detects $l_u(s) = 0$; it ignores level deficiencies. The formulation above generalises it by comparing levels rather than mere presence. For a disjunctive ("A or B") requirement, the best-covered branch is resolved first; the full procedure is given in Algorithm 1 (Section 3.4). For the running example, the computed gap is shown in Table 1.3.

Table 1-3 Compute skill gap.

Skill	Situation	Gap
Spring Boot	held, but below level (1 → 3)	[2, 3]
Microservices	missing entirely	[1, 2]
PostgreSQL | MySQL	satisfied by MySQL	∅
Docker	missing, nice-to-have	[1]

- **Top-K Learning Path (R_u).** a ranked subset $R_u = \{c_1, c_2, \ldots, c_k\}$ that covers the computed gap, with foundational levels placed before advanced ones. A course c_i is considered relevant if it teaches at least one skill in the gap, that is, if

$$S_{ci} \cap Gap \neq \emptyset$$

1.4.3 Research Delimitations

This work covers three things: structured extraction, gap computation, and tree-based retrieval. Several directions are out of scope. The system does not use collaborative filtering, and it ignores historical user behaviour such as clicks or ratings. It is not retrained online from feedback. It also does not integrate directly with third-party platforms such as Udemy or the Coursera player; the course data is collected separately and in advance.

1.5 Thesis Structure

The rest of the thesis is organised as follows. Chapter 2 sets the context, covering language models, tree search, and the recommender systems closest to this one, and explains how the proposed design differs from them. The system itself is the subject of Chapter 3, from the extraction prompts and the skill-gap model to the course tree and the deterministic retrieval and ranking steps. Chapter 4 then turns to the experiments. It measures extraction accuracy, token cost, recommendation quality, and what happens when the LLM is removed from search and ranking. Chapter 5 concludes and points to future work.

CHAPTER 2 PRELIMINARIES AND RELATED WORKS

2.1 Preliminaries

2.1.1 Large Language Models (LLMs) and Prompt Engineering

Large language models are built on the Transformer architecture, and through self-attention they capture context and meaning in text far better than earlier statistical methods. That strength comes with a cost. When a model is pushed into strict information-extraction work, it can return non-deterministic output and invent facts, a behaviour usually called hallucination.

Prompt engineering is the main lever for keeping that behaviour in check. It is the practice of writing the input instructions so that the output is constrained in predictable ways. The thesis uses it to turn a general model into a narrow extraction tool, with three constraints. First, a blacklist tells the model which concepts to drop, such as soft skills or methodologies, which removes noise. The prompt also forces the answer into a fixed JSON shape, so the code that reads it needs no special handling. Finally, the temperature is set to 0, so the same input gives essentially the same output every time.

2.1.2 Tree-based Search and Depth-First Traversal

A tree is a hierarchical structure of nodes joined by edges. It starts from a single root, branches through internal nodes, and ends at leaves. This shape fits categorised data well, and a course catalogue is a good example: a domain such as Backend divides into areas such as Databases, which in turn hold individual courses.

Depth-first search (DFS) is a standard way to walk such a structure. It follows one branch as far as it goes, then backtracks and tries another. Paired with a tree, DFS makes branch pruning possible: if a whole branch can be judged irrelevant to the query, the search skips everything beneath it instead of inspecting each item one by one.

It is worth being precise about what this does and does not buy. Pruning lets the search ignore large parts of the catalogue, which keeps the candidate set small and neatly grouped by domain. By itself, though, a tree does not guarantee a speed advantage

over a flat scan when the matching at each node is cheap; Chapter 4 looks at this question with measurements. In this work the tree is valued chiefly for the structure it gives both the search and the resulting learning path, and the choice of whether to enter a branch is made by deterministic skill-set matching rather than by a model.

2.1.3 In-memory Caching and Short-Circuit Evaluation

Two systems techniques keep the online pipeline responsive. The first is in-memory caching. At start-up the whole catalogue is loaded once from the database into a single in-memory structure, so that lookups during search become plain memory reads instead of repeated database queries. Since the catalogue changes only when new courses are imported, holding it in memory is both safe and quick.

The second is short-circuit evaluation, where a computation stops the moment its result is settled. The pipeline relies on this at several points. If the gap analysis finds that a candidate already meets every requirement, the system returns at once and skips retrieval and ranking entirely. During scoring, if a deterministic check shows that a course does not teach the required skill, the course is rejected immediately, before the remaining criteria are computed. These early exits save work that could not change the outcome.

2.2 Related Works

2.2.1 Traditional Job Matching and Keyword-based NLP

Early automated recruitment matching relied on rule-based string matching and statistical NLP. Applicant-tracking systems compared a CV against a JD using regular expressions, Jaccard similarity, or TF-IDF weighting.

These methods share one weakness: they are semantically blind. Because they treat text as a set of tokens rather than as connected meaning, a keyword system cannot see that “C# or Java” is an alternative rather than two mandatory skills, and it cannot tell a beginner’s “exposure to Python” apart from a senior engineer’s “Python architecture.” The result is a high rate of false matches and weak precision in technical domains.

2.2.2 AI-driven Recommendation in EdTech and LLMTreeRec

As deep learning matured, EdTech platforms such as Coursera, Udemy, and edX moved toward neural recommenders. Most rest on collaborative filtering, matrix factorisation, or content-based networks that predict how likely a user is to engage with or complete a course.

A more recent and directly relevant line of work is LLMTreeRec, published at COLING 2025. It organises a large item catalogue as a tree and uses a language model to navigate it with depth-first search, letting the model decide, branch by branch, where to descend. This avoids scanning the whole catalogue and shows that pairing a hierarchy with model-driven routing can keep retrieval efficient on very large item sets.

Its routing, however, is driven by historical user behaviour such as click logs and ratings. That works for commercial engagement, but it says little about whether a particular candidate actually holds the skills a job demands, and it does not address the cold-start case, where a person has to be assessed against objective requirements rather than against a record of past activity.

2.2.3 Limitations of Prior Work and the Position of This Thesis

This thesis keeps the part of LLMTreeRec that works well, the idea of organising courses as a hierarchy, but it rebuilds the reasoning around that hierarchy. The differences can be set out as four points.

First, retrieval is deterministic rather than model-driven. Where LLMTreeRec asks the model at each branch whether to descend, this system precomputes, for every node, the set of skills taught beneath it, and decides whether to enter a branch by intersecting that set with the candidate's skill gap. The decision is exact, costs nothing, is identical on every run, and cannot hallucinate. In this sense the system is a deterministic variant of LLM-guided tree retrieval: it trades the model's semantic flexibility at the branch for correctness and reproducibility.

Second, the input is competency, not behaviour. Instead of relying on click history, the system extracts both the skill and its required level directly from the CV and the JD, which lets it serve a cold-start user who has no interaction history.

Third, the gap is quantified. A dedicated module computes the exact distance between what a job needs and what a candidate has, turning free text into a structured list of missing skills and the levels still to be reached.

Fourth, the recommended path is level-progressive. Rather than ordering courses by popularity, the system resolves the gap in order, placing foundational courses before advanced ones so the learner builds up in a sensible sequence.

Taken together, these choices move the design away from behaviour-driven popularity and toward a competency-driven, reproducible recommendation, while keeping the language model confined to the one task that genuinely needs it.

CHAPTER 3 A HYBRID FRAMEWORK FOR SKILL GAP ANALYSIS AND COURSE RECOMMENDATION

3.1 System Overview

The system is built using a multi-stage process.

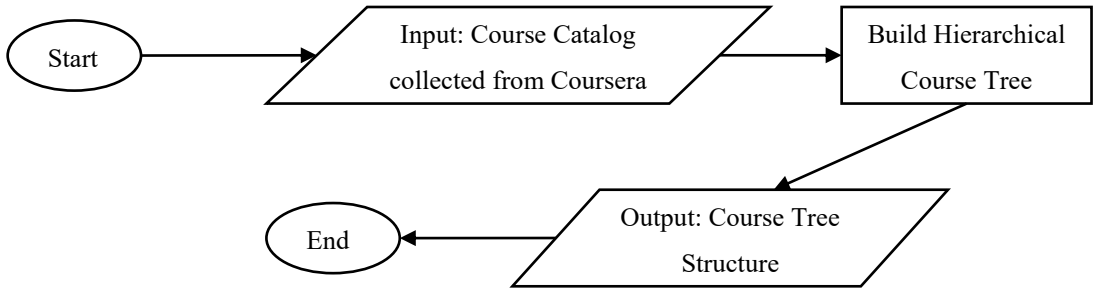


Figure 3-1 Build Hierarchical Course Tree.

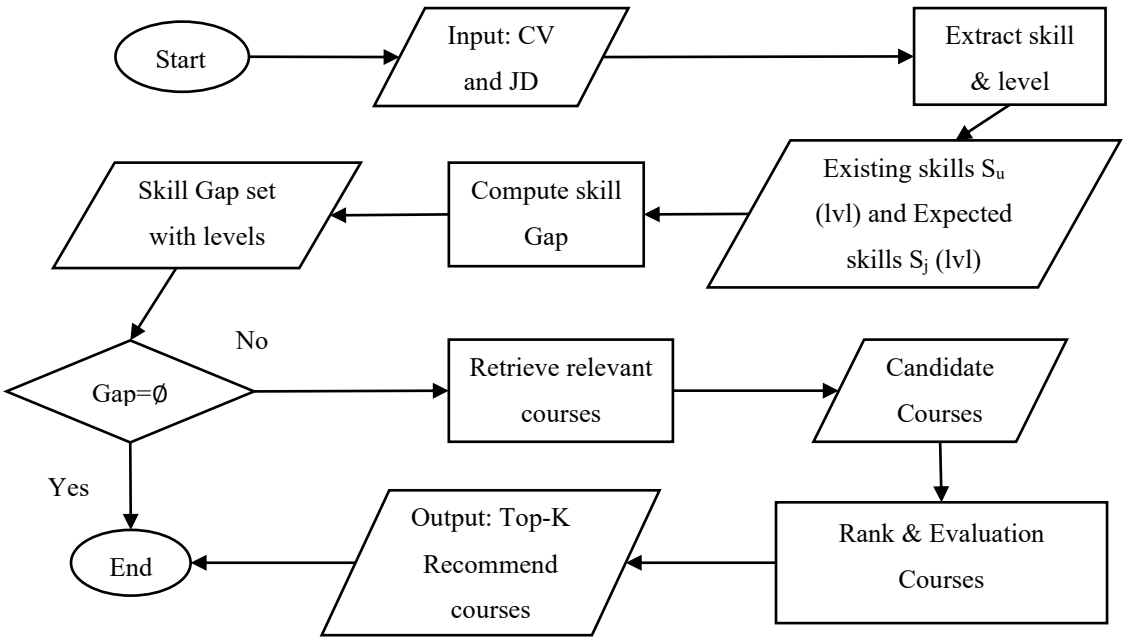


Figure 3-2 Overall Flowchart.

Based on the overall flowchart, the execution flow of the system consists of two main phases: the Offline Pre-processing Phase and the Online Matching Phase, which are further divided into four core modules:

3.1.1 Offline Phase: Hierarchical Course Tree Construction

Searching through thousands of courses at once is very difficult, so course collection and tree construction will be done beforehand. Course information is collected, cleaned, and categorised, then pushed into a hierarchical tree structure in memory. This serves the online search phase.

3.1.2 Online Phase: The Recommendation Pipeline

The online phase runs when a user submits a CV and a target JD. It proceeds in five steps.

1. **Skill and level extraction.** A language model reads the CV and the JD and returns two structured sets: the candidate's existing skills (S_u) and the job's required skills (S_j), each with a level.
2. **Skill gap computation.** Su and Sj are compared to produce the gap: the skills the candidate is missing, together with the levels still to be reached. If the gap is empty, the candidate already meets the requirements and the pipeline stops here.
3. **Relevant course retrieval.** A depth-first search over the course tree, guided by deterministic skill-set matching, collects the candidate courses without scanning the whole catalogue.
4. **Relevance scoring.** Each candidate course is scored by a deterministic function $E(c)$ that checks skill match, level alignment, and redundancy.
5. **Top-K selection.** The scored courses are ordered into a learning path. Courses that cover several gaps at once are preferred, and the system returns the Top-K as the final recommendation.

3.2 Offline Phase: Building the Hierarchical Course Tree

The quality of the recommendations depends directly on the quality of the catalogue, so the offline phase is where most of the data work happens. It has two parts: a data pipeline that produces clean course records, and a tree builder that organises them.

3.2.1 Data Pipeline

The pipeline turns raw web pages into clean, structured course records through four steps.

Crawling. Courses are scraped from Coursera with Selenium driving a headless browser. To behave like a real visitor, the crawler scrolls smoothly and stops once four scroll cycles bring no new items. It collects the title, the URL, the level inferred from Beginner/Intermediate/Advanced labels, and the listed skills. Footer and "related courses" blocks are removed with BeautifulSoup before parsing, and a hash set of previously seen URLs lets the crawler skip courses it has already collected.

Skill cleansing. Skills scraped from course pages carry a lot of noise, such as the words learn, review, or hour. A multi-stage filter handles this. Core skills with special characters (C#, .NET, React) are protected by a lookup table so a regular expression cannot truncate them. Variants are then normalised to a single name (react.js and react native both become ReactJS). Finally a blacklist removes raw tools (vscode, maven) and vague academic phrases (software engineering, best practices).

Miss checking. To make sure the filter does not throw away useful skills, a separate tool counts how often each rejected keyword appears and produces an audit report, which is used to refine the rules over time.

Normalisation and import. The cleaned courses are written to SQL Server. Before inserting new skill links for a course, the old ones are deleted, which keeps the data consistent. After a successful load the script calls a reload endpoint on the server, so the in-memory tree is rebuilt without a restart.

3.2.2 Hierarchical Tree Construction

Once the data is clean, it is organised into a tree. The schema is fixed in advance: eleven top-level domains, each holding two to seven skill groups. For instance, Backend contains groups such as Java Ecosystem, C# / .NET, and NodeJS. Fixing the schema keeps the tree bounded and stops the classifier from inventing endless new branches.

Each course is placed in the tree by a classifier. Rather than clustering, which struggles with short technical text, the system sends the title and cleaned skills to the

LLM and asks for two fields, category and subcategory, drawn only from the fixed schema. A regular expression isolates the JSON block in case the model adds extra text. Classification is parallelised with a thread pool, and because the query only selects unclassified courses, an interrupted run resumes where it left off. When the model returns a label outside the schema, the course is marked for manual review instead of being forced into a wrong branch.

At start-up the catalogue is loaded into RAM. A single query concatenates each course's skills with STRING_AGG, which avoids nested joins, and the flat result is shaped into a nested dictionary with three tiers: the eleven domains at the top, the skill groups beneath them, and, at the leaves, the courses themselves. Each leaf course carries its full payload (identifier, title, skills, level, description), so later stages never need to query the database again.

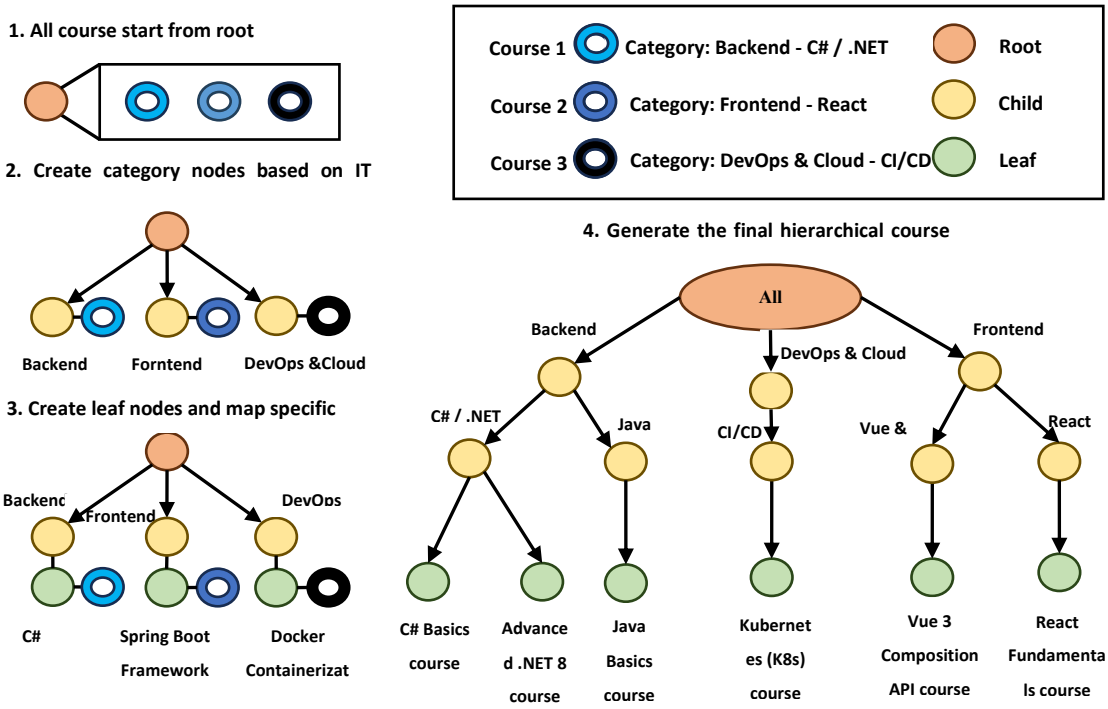


Figure 3-3 Course tree structure

3.3 Skill & Level Extraction

Extraction turns the unstructured text of a CV or JD into a structured set of skills with levels. This is the one stage that relies on the language model, because reading natural language is exactly what keyword rules cannot do. The module combines light pre-processing, a tightly constrained LLM call, and a normalisation step.

3.3.1 Pre-processing

Job descriptions vary a lot in formatting, which can confuse the model, so the text is tidied first. Separators that imply a choice, such as a slash or the words “or” and “and/or”, are normalised to a single form so the model can spot alternatives. Curly braces are escaped to avoid format-string errors in Python. The input is also truncated at 8,000 characters to keep latency and memory in check.

3.3.2 Prompt Design and LLM Extraction

The system utilises a Cloud-based LLM (Qwen-480B) via API, assigning the model the role of a core ATS engine. Instead of allowing the LLM to generate text freely, the system shapes the output using Strict Rules and enforces a mandatory JSON format. The extraction process applies the following strategies:

3.3.2.1 Zero-Tool & Blacklist Rule:

The system forces the LLM to focus exclusively on Hard Tech Skills.

- ***Elimination of soft skills and methodologies****:* Keywords such as Agile, Scrum, DevOps, and Communication are **discarded** as they do not contribute to constructing a specific programming learning path.
- ***Raw Tools Filtering: Specific utility tools*** (e.g., GitLab CI, Jenkins, Postman, npm) are omitted to retain Core Concepts like CI/CD and REST API, thereby improving the accuracy of subsequent course matching.
- ***Anti-Hallucination****:* The LLM is strictly prohibited from inferring or automatically adding programming languages that do not explicitly appear in the original text.

3.3.2.2 Propositional Logic Extraction:

This is a core technique for handling the structural complexity of Job Descriptions. The system guides the LLM to identify and flag each skill via the Match_mode parameter:

- ***Independent Clauses (EXACT):*** Assigned to standalone or regularly listed technologies. The candidate is strictly required to possess this skill.
- ***Disjunctive Clauses (ANY):*** When the JD requests one of multiple options (e.g., C#.NET Core or Java), the LLM must merge these skills into a single

string separated by a pipe character (|) and assign the ANY label (e.g., C#.NET Core|Java).

- ***Parentheses Rule for Nested Groups:*** If the text contains a broad concept accompanied by a list of tools in parentheses (e.g., AI tools (Copilot, ChatGPT)), the system instructs the LLM to ignore the external general keyword and extract only the internal tools, grouping them under an ANY label (Copilot|ChatGPT).

3.3.2.3 Level Quantification:

Each extracted skill is also mapped to a level from 1 to 3, inferred from the wording around it, as summarised in Table 3.1.

Table 3-1 Mapping from job-description wording to proficiency level.

Level	Assigned when the wording indicates
Level 3 (Master)	deep expertise ("proficient", "expert", "master") or ≥ 3 years of experience
Level 2 (Standard)	standard proficiency ("experienced", "good working knowledge") or 1–2 years; this is the default level
Level 1 (Basic)	a skill mentioned only briefly, or listed under "Preferred" or "nice to have"

3.3.2.4 JSON Slicing Mechanism:

To prevent errors caused by the LLM generating unexpected extraneous text (Hallucination) that could corrupt the JSON parsing process, the system employs a primitive index-seeking technique (result_text.find('[') and result_text.rfind(']')). This technique forces the source code to extract exclusively the valid JSON core, so stray text outside the array does not break parsing.

3.3.3 Global Alias Mapping

Raw model output still suffers from fragmentation: C#.NET, .NET 8, and C# mean the same thing but are written differently. A static alias map resolves this. After lowercasing and stripping noise characters, every skill is looked up in the map and converted to one canonical name, so that react and react.js both resolve to reactjs.

This normalisation is what lets the gap computation in the next section compare skills reliably.

3.4 Computing skill gap

With the CV and JD reduced to structured skills, the system computes the gap: the technologies the candidate either lacks entirely or holds at too low a level. Let Sj be the skills the job requires and Su the skills the candidate has. The gap is built in three steps.

3.4.1 Dynamic Alias Resolution

A single technology is often written in several ways, for example REST API, RESTful API, or Web API. To avoid mismatching on wording alone, the matcher expands each target skill through the alias map and compares canonical forms, combined with safe token-level matching. This way the candidate's competency is recognised regardless of how it was written in the CV.

3.4.2 Disjunctive Skill Groups

For skills with optional characteristics (Match_mode: **ANY**), the candidate is not required to fulfill the entire list. The system splits the skill cluster (e.g., MySQL|PostgreSQL|Oracle) into independent branches (Candidates). For each branch $\boldsymbol{c}$, the system evaluates the suitability score $Score(c)$ based on the total fulfillment level of its sub-skills $\boldsymbol{s} \in \boldsymbol{c}$:

$$Score(c) = \sum_{s \in c} f(s)$$

Where the function $f(s)$ evaluates the user's current level (l_{user}) against the required level (l_{req}):

- $f(s)$ = 1 if l_{user} >= l_{req} (Fully meets the requirement)
- $f(s)$ = 0.5 if 0 < l_{user} < l_{req} (Possesses foundational knowledge but lacks depth)
- $f(s)$ = 0 if l_{user} = 0 (Completely unknown)

The system automatically prioritises the branch with the highest total $Score(c)$. The algorithm's distinctive feature lies in its tie-breaking mechanism: If multiple branches

achieve the same score (Tied Score), the system applies the "Shortest Path" principle by comparing the length of the missing skills array (len(branch_missing)). The branch requiring the fewest remedial courses is prioritized to optimize the candidate's training time.

Furthermore, for complex nested structures (e.g., Java (Spring Boot, Hibernate)), the system triggers the parse_complex_candidate mechanism using Regular Expressions (Regex) to independently extract the base language and its accompanying frameworks. This ensures the matching process exhaustively evaluates all component technologies without generating a "Phantom Gap". For a requirement marked ANY, the candidate need not satisfy the whole list. The system splits the group (for example MySQL | PostgreSQL | Oracle) into separate candidate branches and scores each branch by how well its sub-skills are met:

$$f(l_u, l_{req}) = 1 \; if \; l_u \geq l_{req}, \qquad 0.5 \; if \; 0 < l_u < l_{req}, \qquad 0 \; if \; l_u = 0$$

The branch with the highest total score is chosen. If two branches tie, the system applies a shortest-path rule and prefers the one with fewer missing skills, which keeps the resulting learning path shorter. For nested groups such as Java (Spring Boot, Hibernate), a small parser extracts the base language and its frameworks separately, so every component is evaluated and no phantom gap is created.

3.4.3 Progressive Level Array Calculation

Unlike systems that report a skill as simply met or not met, this work treats the gap as a sequence of levels to climb. Suppose a skill is required at level l_req while the candidate sits at l_u, with l_u < l_req. The gap for that skill is the array of intermediate levels still to be acquired.

For example, if a JD asks for C# at level 3 but the CV shows only level 1, the gap is the array [2, 3]: the learner should complete level-2 material before moving on to level 3. This progressive array gives the later retrieval and ranking steps a precise target to work against.

Algorithm 1. Progressive Skill Gap Computation.

```
Input : Su (candidate skills with level), Sj (required skills,
mode EXACT or ANY)
```

```
Output: Gap (missing skills with progressive levels)
1  Gap ← ∅
2  for each skill s in Sj do
3      if s.mode == EXACT then
4         l_user ← level(s, Su)       // via find_matching_skill
5          l_job  ← level(s, Sj)
6          if l_user < l_job then
7              Levels ← [max(1, l_user+1) ... l_job]
8              Gap ← Gap ∪ { (s, Levels) }
9      else if s.mode == ANY then
10         Cands ← split branches of s on '|'
11         best ← null ; max_score ← -1 ; min_missing ← ∞
12         for each branch c in Cands do
13             Sub ← parse_complex_candidate(c)
14             score ← 0 ; missing ← 0
15             for each sub in Sub do
16                 if level(sub, Su) ≥ level(sub, Sj) then score
← score + 1
17                else if level(sub, Su) > 0 then  score ← score
+ 0.5 ; missing ← missing + 1
18                 else missing ← missing + 1
19              if score > max_score or (score == max_score and
missing < min_missing) then
20                  max_score ← score ; min_missing ← missing ;
best ← c
21         Levels ← progressive missing levels of best
22         if Levels ≠ ∅ then Gap ← Gap ∪ { (s, Levels) }
23 return Gap
```

3.5 Course Retrieval by Deterministic Tree Search

3.5.1 Problem and Approach

Given a skill gap, the system must find courses that teach the missing skills. Querying a large flat catalogue directly runs into two problems. Keyword matching is semantically blind, and calling a language model for every record is far too slow and expensive to be practical. The approach taken here organises the catalogue as a tree and decides which branches to explore using deterministic skill-set matching, so that retrieval needs no model calls at all.

This is a deliberate change from the LLM-routed style of tree retrieval. Instead of asking a model, branch by branch, whether to descend, each node stores the set of

skills taught beneath it, computed once when the tree is built. Deciding whether to enter a branch is then a set intersection between the gap and that precomputed set. The decision is exact, costs nothing, and is the same on every run. Chapter 4 reports the effect of this choice on recall, latency, and cost.

3.5.2 Depth-First Traversal

Retrieval walks the tree with depth-first search, using a stack of nodes still to visit. Each stack entry carries both the node's contextual path (for example, "Backend → Java Ecosystem") and the node's content. Keeping the path preserves the hierarchical context, which is useful for presenting the learning path by domain later.

3.5.3 Deterministic Skill-set Pruning

The heart of the method is how a branch is judged. When the tree is built, every category and subcategory node is annotated with the union of the skills taught by the courses below it. During search, a branch is relevant if the gap and the node's skill set share at least one skill. If they do, the search descends; if not, the whole branch is pruned and never examined further.

Because this test is a plain set operation, it needs no cache of model decisions and no fallback for network failures: there is no network call to fail. An earlier version of the system asked the model a yes/no question at each node and cached the answers, but that mechanism was removed once the skill-set test made it unnecessary. Removing it also removed the model's cost and its run-to-run variation from the retrieval stage. Algorithm 2 states the procedure.

Algorithm 2. Deterministic DFS Tree Search with Skill-set Pruning.

```
Input : T (course tree in RAM, each node annotated with its skill
set),
         G (candidate's missing skills)
Output: Candidates (list of candidate courses)
1  Stack ← ∅ ; Candidates ← ∅
2  for each (cat_name, cat_node) in T do  Stack.push((cat_name,
cat_node))
3  while Stack ≠ ∅ do
4      (path, node) ← Stack.pop()
5      // deterministic relevance: do the gap and the node share
a skill?
```

```
6       if G ∩ node.skills ≠ ∅ then
7           if node is a leaf then
8               Candidates.extend(node.courses)
9           else
10              for each (sub_name, sub_node) in node do
11                Stack.push((path + ' → ' + sub_name, sub_node))
12      else
13           // prune: skip the entire branch (no model call, no
cache)
14 return Candidates
```

3.5.4 Retrieval Stage Results

The traversal returns the courses under every branch whose skill set overlaps the gap. The size of this candidate pool depends on the gap. For a narrow gap it is small, but for a gap containing common skills such as Linux or CI/CD it can cover most of the catalogue, because those skills appear across many branches. The pool is therefore best understood as a structured, deduplicated candidate set rather than a small fixed shortlist; the filtering and ranking that follow are what reduce it to the final recommendation.

3.6 Deterministic Ranking and the Evaluator

Retrieval produces a candidate pool; ranking turns it into an ordered learning path. This stage is deterministic throughout. It selects combo courses that cover several gaps at once, orders the rest by a relevance score, and audits every choice with the evaluator E(c).

3.6.1 Combo Courses via Set Cover

Before building a step-by-step path, the system looks for combo courses, that is, single courses that close more than one gap at once and so save the learner time. For a course c and the missing-skill set G, the coverage is the size of their intersection:

$$coverage(c) = |skills(c) \cap G|$$

Skills are compared through the same safe matching used elsewhere, so wording differences do not hide a match. Courses whose coverage exceeds a threshold are pulled out, scored by coverage, and placed near the top of the recommendation. This

is a practical take on the set-cover idea: prefer the few courses that cover the most of the gap.

3.6.2 Deterministic Relevance Ranking

For the remaining single-skill gaps, the candidate pool is grouped by skill-and-level milestone, and one course is chosen for each milestone. An earlier design asked the language model to make this choice, in chunks of a few milestones at a time. That design was replaced, for two reasons that Chapter 4 documents: the model's choice was not reproducible, changing between runs, and it did not improve quality over a well-designed deterministic rule.

The current ranker scores each candidate for a milestone by three deterministic signals, in order of weight. The strongest signal is whether the gap skill appears in the course title, which is the most reliable evidence that the course is really about that skill. Next is coverage, the number of gap skills the course matches. Last is specificity, the share of the course's skills that are relevant to the gap, which favours focused courses over broad ones. The course identifier breaks any remaining tie, which makes the ordering completely reproducible. The top-scoring course is taken for each milestone.

3.6.3 The Evaluator E(c)

Every selected course passes through a deterministic evaluator before it is returned. The score combines three terms:

$$E(c) = w^1 \cdot SkillMatch(c) + w^2 \cdot LevelAlign(c) - w^3 \cdot Redundancy(c),$$
$$with\ w^1 = 0.5, w^2 = 0.4, w^3 = 0.1$$

SkillMatch(c) is 1 when the course actually teaches the target skill, checked with safe substring and alias matching, and 0 otherwise. LevelAlign(c) is 1 when the course level matches the required level, 0.5 when it is a foundational review below that level, and 0 when it overshoots. Redundancy(c) is the fraction of the course's skills that are irrelevant to the gap, which penalises bloated courses. A course is kept only if E(c) clears a threshold of 0.7; a course that fails the skill check (SkillMatch = 0) cannot reach that threshold and is dropped.

In the current architecture E(c) acts mainly as a safety check rather than an active filter, because every candidate has already passed deterministic skill-and-level filtering before it is scored. Chapter 4 reports an ablation confirming this: removing E(c) does not change the results on the test set. The evaluator therefore matters as a guard that would catch a regression if an earlier stage were loosened, not as a component that reshapes the output today.

3.6.4 Quality Metrics: Precision@K and Coverage

Two metrics summarise the quality of a learning path, and both are computed by deterministic code rather than judged by the model. Precision@K is the fraction of the top K recommended courses that pass an independent relevance check:

$$Precision@K = (relevant\ courses\ in\ the\ top\ K) \ / \ K$$

Skill Coverage is the fraction of the gap that the path actually addresses:

$$Coverage = (unique\ gap\ skills\ taught\ by\ the\ path) \ / \ (total\ gap\ skills)$$

Relevance here is decided by exact substring and alias matching against the gap, with the model excluded from the check, so the metric cannot be inflated by the same model that produced the recommendations. The values these metrics take on the test data are reported in Chapter 4 rather than asserted here.

CHAPTER 4 EXPERIMENTS AND EVALUATIONS

This chapter empirically evaluates the proposed system. Rather than reporting a single end-to-end accuracy figure, the evaluation is decomposed into five focused experiments (P1–P5), each isolating one design decision: where the system relies on the Large Language Model (LLM) and where it relies on deterministic computation. The guiding question is not merely "how accurate is the system," but "is the LLM placed only where it adds measurable value, and is the rest of the pipeline reproducible?" All metrics are computed by deterministic Python scripts against a manually labelled ground truth; the LLM is never used to score its own output.

4.1 Experimental Setup

4.1.1 Datasets

Two categories of data were prepared: the educational supply (courses) and the labor demand (CV–JD pairs). The course catalogue was crawled from Coursera and passed through the cleansing pipeline described in Chapter 3, yielding **5,856** courses organised into 11 categories and 45 subcategories. To evaluate the recommendation pipeline, 100 synthetic CV–JD pairs were constructed spanning 15 technical domains (e.g., Backend Java, Frontend React, DevOps, Data/AI, Security) and three gap types (missing-skill, missing-level, and disjunctive "A-or-B" requirements).

Because the recommendation and extraction tasks require different ground truth, two manually labelled subsets were created. For the retrieval and ranking experiments (P3–P5), a balanced subset of **36 CV–JD pairs** (12 per gap type, covering all 15 domains) was labelled at the course level: each candidate course surfaced by the system, plus a random sample drawn from the matching pool, was annotated as relevant (1) or not relevant (0). For the extraction experiment (P1), the required skills of **36 job descriptions** were independently annotated as a gold skill set. Labelling followed a fixed protocol: the title of the course (or the explicit wording of the JD) was treated as the primary evidence; an AI tool was permitted to assist analysis, but the final decision on every label was made by the author. A second independent annotation for inter-rater agreement is identified as desirable future work.

Table 4-1 Summary of datasets used in the evaluation.

Dataset	Description
Course catalogue	5,856 cleansed Coursera courses; 11 categories; 45 subcategories
CV–JD test set	100 synthetic pairs; 15 domains × 3 gap types
Recommendation ground truth (P3–P5)	36 balanced pairs, course-level relevance labels (595 labels: 200 relevant, 395 not)
Extraction ground truth (P1)	36 JDs with manually annotated gold skill sets (131 gold skills)

4.1.2 Implementation Environment

The backend is implemented in Python with FastAPI. The course catalogue is stored in Microsoft SQL Server and loaded once into an in-memory tree at start-up. The single LLM component (skill extraction) uses Qwen3-Coder-480B accessed through the Ollama cloud endpoint with temperature fixed at 0 and JSON-schema-constrained prompting. All deterministic components tree search, gap-based filtering, ranking, and the evaluator E(c) run as plain Python with no network dependency, which is what makes the non-extraction stages fully reproducible.

4.1.3 Evaluation Protocol and Metrics

The five experiments and the quantity each isolates are summarised in Table 4.2. Skill-set quality is measured with standard Precision, Recall, and F1. For a labelled item set, Precision = TP / (TP + FP), Recall = TP / (TP + FN), and F1 = 2·P·R / (P + R), where TP, FP, and FN denote true positives, false positives, and false negatives against the human labels. Reproducibility is measured by running a deterministic stage multiple times and reporting the average pairwise Jaccard overlap of its outputs (1.0 = identical every run).

Table 4-2 The five evaluation experiments and their purpose.

Exp.	Question	What it isolates
P1	LLM vs. keyword skill extraction	Justifies keeping the LLM at extraction
P2	Where does token reduction come from?	Separates the filter from the tree
P3	End-to-end recommendation quality	Real P/R/F1 + value of guardrail E(c)
P4	Pruning: deterministic vs. LLM	Justifies removing the LLM from search
P5	Ranking: deterministic vs. LLM	Justifies removing the LLM from ranking

4.2 Skill Extraction Accuracy (P1)

Skill extraction is the only stage that still relies on the LLM, so it must be shown to outperform a non-LLM alternative. The proposed extractor (Qwen3-Coder-480B with strict prompting) is compared against a keyword baseline that scans the job description for entries from the system's own skill vocabulary using word-boundary matching. Crucially, the baseline is given the same vocabulary the system knows, so the comparison isolates semantic ability (handling "A or B" clauses, proficiency levels, and parenthetical tool groups) rather than dictionary coverage. Following the supervisor's guidance, raw counts are reported before percentages.

Table 4-3 Skill extraction: raw counts and Precision/Recall/F1 over 36 JDs (131 gold skills).

Method	Gold	Extracted	Correct	Precision	Recall
Keyword baseline	131	132	111	84.1%	84.7%
LLM (Qwen3-Coder-480B)	131	140	131	93.6%	100.0%

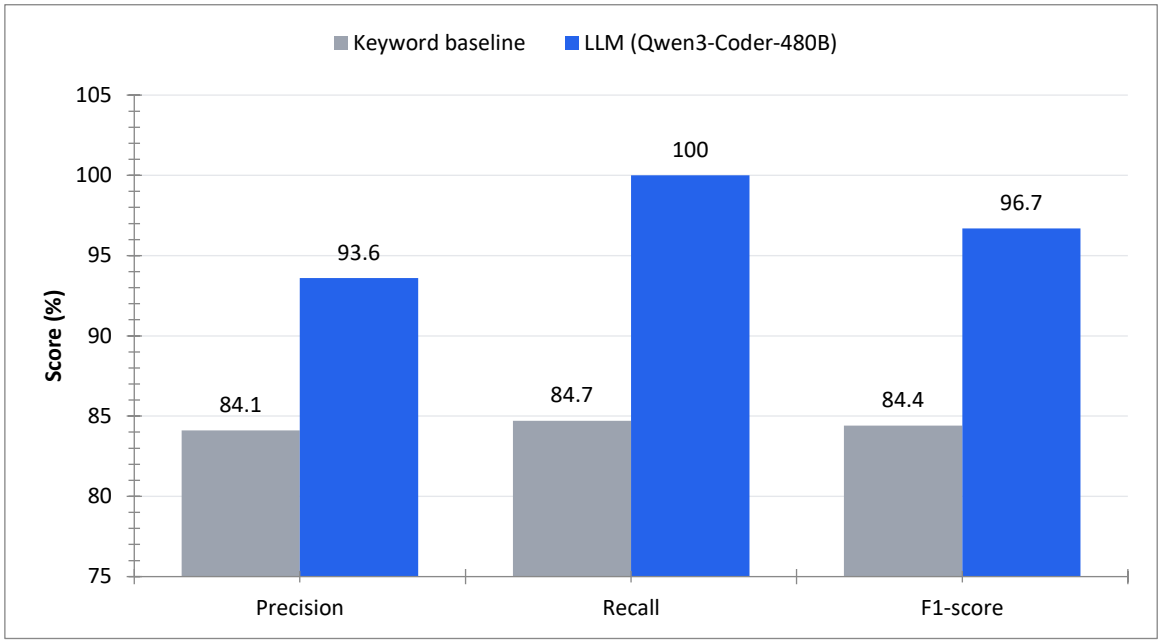


Figure 4-1 Precision, Recall, and F1 of keyword baseline versus LLM extraction.

Figure 4.1 visualises the result. The LLM reaches an F1 of 96.7% (P = 93.6%, R = 100.0%) against the baseline's 84.4% (P = 84.1%, R = 84.7%). It is important to report this honestly: the keyword baseline is not useless a good static dictionary recovers most flat skill mentions. The LLM's advantage is concentrated in three specific abilities.

- **Higher precision:** the baseline over-extracts (21 false positives), for example treating a role title such as "DevOps" as a skill, whereas the LLM filters such noise.
- **Disjunction handling:** across the 10 JDs containing an "A or B" requirement, the LLM grouped all 10 correctly (10/10), while the keyword baseline grouped none (0/10) because it cannot recognise that the two options are alternatives rather than two mandatory skills.
- **Level quantification:** the LLM assigned the correct proficiency level (1–3) for 83.2% of skills (109/131); the keyword baseline cannot produce levels at all.

These three capabilities not raw skill spotting are what justify retaining the LLM at the extraction stage even though every later stage is deterministic. The 9 LLM false positives (precision 93.6%, not 100%) reflect occasional leakage of out-of-scope or title-derived terms and are reported as a known imperfection rather than concealed.

4.3 Token Consumption Analysis (P2)

A central claim about hierarchical retrieval is that it reduces the volume of data sent to the LLM. This experiment tests where that reduction actually comes from by measuring, over all 100 test cases, the tokens that would be sent to an LLM ranker under three configurations: feeding all courses (Naive), the deterministic gap filter applied to a flat scan (Filter-only), and the full pipeline (Tree + Filter). Results are summarised in Table 4.4 and Figure 4.2.

Configuration	Avg. tokens	Reduction vs. Naive
Naive (all 5,856 courses)	434,029	—
Filter-only (deterministic)	1,052	99.8%
Tree + Filter (full pipeline)	1,052	99.8%

Table 4-4 Average tokens sent to the LLM per query (mean over 100 cases).

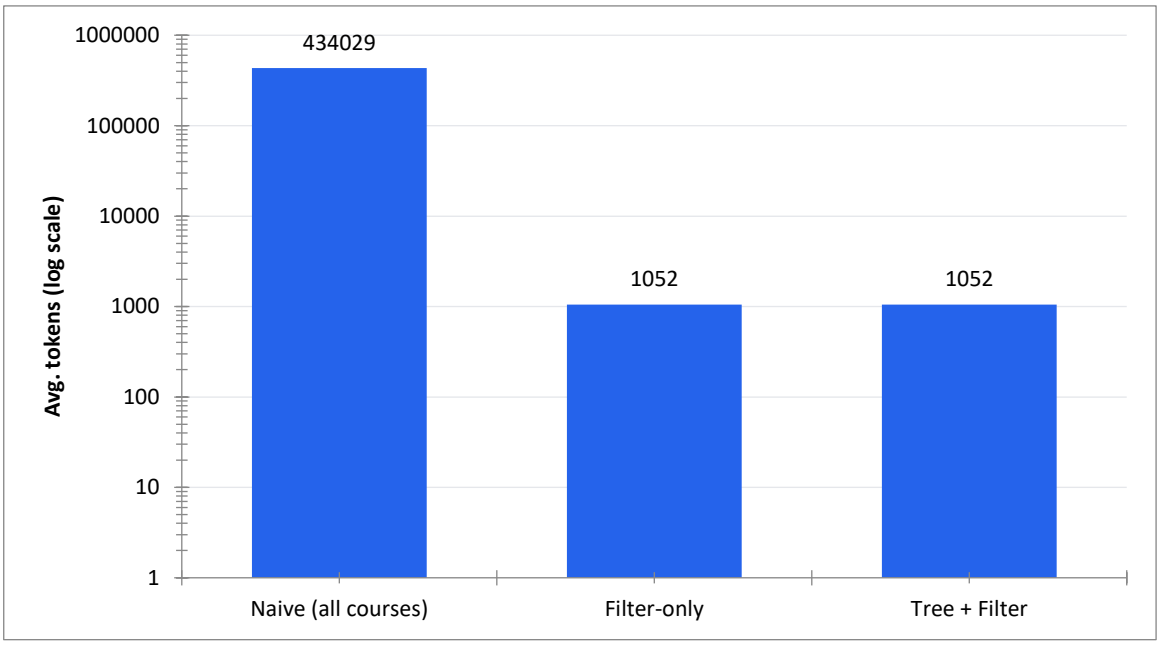


Figure 4-2 Token consumption per query on a logarithmic scale

Figure 4.2 makes the source of the reduction explicit. The deterministic gap filter cuts token volume by 99.8% (434,029 → 1,052). Adding the tree on top of the filter contributes a further 0.0%: the tree returns the same set of courses as the flat scan, so after filtering the two configurations are token-for-token identical on every task. The honest conclusion, which differs from an intuitive “tree makes it cheaper” narrative, is that the token saving is produced by the deterministic filter; the tree’s role is

organisational (domain-structured learning paths, maintainability, and a basis for optional semantic routing), not token reduction.

4.4 Recommendation Quality and Guardrail Ablation (P3)

This experiment measures the quality of the final recommendations against the 36 course-level labelled pairs, and ablates the deterministic evaluator E(c) to test how much the guardrail contributes. A recommended course is a true positive only if the human annotator labelled it relevant. Results are reported in Table 4.5.

Table 4-5 Recommendation quality with and without the guardrail E(c) (36 pairs).

Configuration	Precision	Recall	F1	TP / FP / FN
With guardrail E(c)	74.8%	61.0%	67.2%	122 / 41 / 78
Without guardrail E(c)	74.8%	61.0%	67.2%	122 / 41 / 78

The system attains a Precision of 74.8%, a Recall of 61.0%, and an F1 of 67.2%. Two points require honest interpretation. First, Precision is the primary metric here it reflects the quality of what the learner is shown and 74.8% indicates that roughly three of every four recommended courses are judged relevant. Second, the lower Recall is a property of a Top-K recommender rather than a defect: the system deliberately surfaces only a few courses per milestone, so it cannot recall every relevant course present in the labelled sample, and Recall is estimated over that sample (recommended items plus a random pool sample) rather than the full catalogue.

The guardrail ablation is the more interesting finding. Disabling E(c) changes Precision by exactly 0.0 percentage points; the two configurations are identical. This is not a copy artefact the two outputs are produced by different code paths but a genuine result: because every candidate has already passed deterministic skill-and-level filtering before E(c) is computed, every course scores above the 0.7 threshold, so the guardrail never rejects anything. E(c) is therefore redundant in the current architecture. It would act as a safety net only if an upstream stage were loosened; with deterministic filtering in place it does not alter the result. Reporting this redundancy is preferable to claiming a guardrail benefit that the data does not support.

4.5 Retrieval (Pruning) Strategy Ablation (P4)

This experiment validates the decision to remove the LLM from tree search. Three pruning strategies are compared on the 36 labelled pairs: an LLM-by-name strategy in the style of LLM-routed tree retrieval (the LLM is asked, per node, whether a branch is relevant); the proposed deterministic skill-set strategy; and a hybrid that uses deterministic pruning but invokes the LLM only to rescue a category the deterministic rule would discard. Recall, average LLM calls, and average latency are reported in Table 4.6 and Figure 4.3 and 4.4.

Table 4-6 Pruning strategy ablation (averages over 36 pairs).

Strategy	Recall	LLM calls	Latency (ms)
LLM-by-name	63.3%	24.1	39,131.9
Deterministic	99.2%	0.0	1,554.4
Hybrid	99.2%	0.3	2,182.5

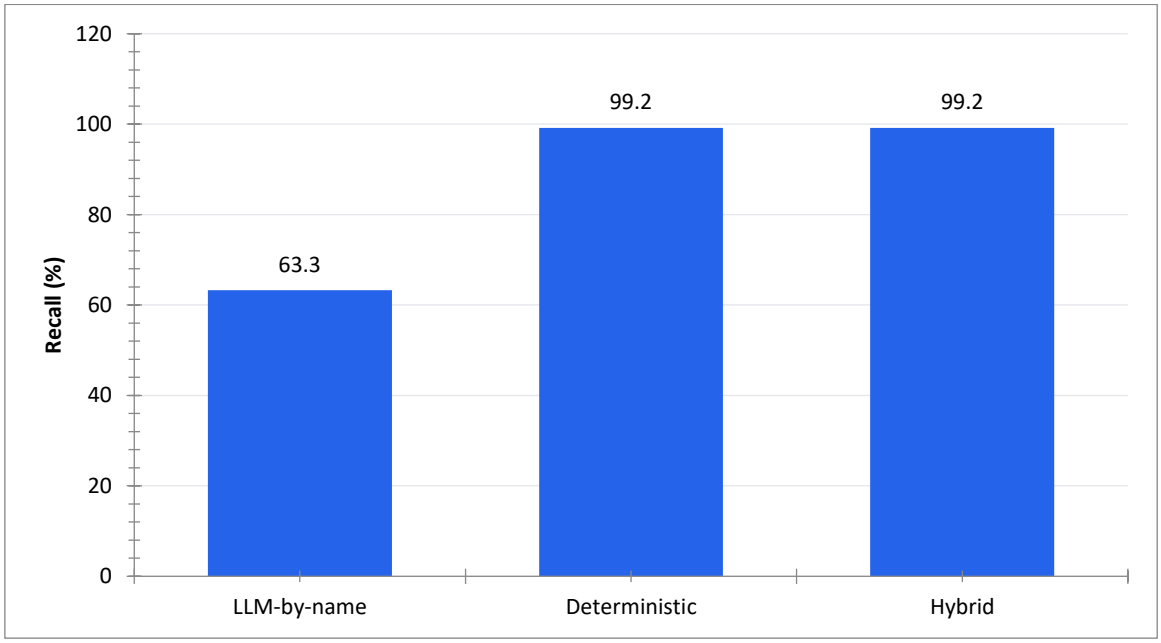


Figure 4-3 Retrieval recall for the three pruning strategies.

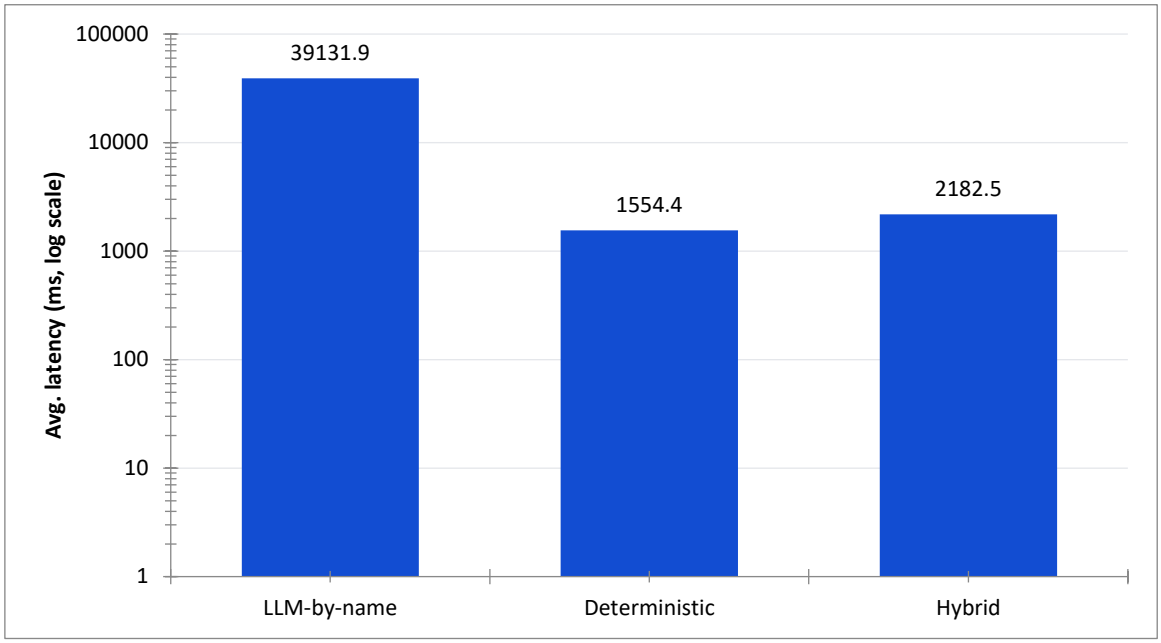


Figure 4-4 Latency for the three pruning strategies

The deterministic strategy dominates on every axis. It achieves 99.2% recall with zero LLM calls and the lowest latency (≈1.55 s), whereas the LLM-by-name strategy reaches only 63.3% recall at roughly 25× the latency (≈39 s) and 24 LLM calls per query. The LLM-by-name approach loses recall because the model occasionally answers "no" for a genuinely relevant branch, pruning courses that should have been kept. This reverses the usual assumption: the deterministic variant is not a compromise that trades quality for speed it is simultaneously more accurate, faster, and free.

The hybrid strategy matches the deterministic recall (99.2%) at the cost of 0.3 LLM calls. Reported honestly, on this test set the LLM rescue did not recover any additional relevant course; the hybrid therefore behaves as a safety net that did not need to fire, and the pure deterministic strategy is sufficient.

4.6 Ranking Strategy Ablation (P5)

The final experiment validates the decision to make ranking deterministic. From the deterministic shortlist of each milestone, one course is selected by three strategies: a naive deterministic baseline that picks the lowest course ID (By-ID); the LLM choosing one course per milestone (the previous design); and the proposed smart deterministic ranker (Smart-Det) that scores courses by title match, gap coverage, and

specificity. Precision is measured over the 36 pairs; reproducibility is measured by repeated runs (pairwise Jaccard) on a 6-task sample to limit LLM cost. Results appear in Table 4.7 and Figure 4.4.

Table 4-7 Ranking strategy ablation: precision, reproducibility, and LLM cost.

Strategy	Precision	Reproducibility (Jaccard)	LLM calls
By-ID (naive deterministic)	43.2%	1.000	0
LLM (previous design)	73.8%	0.938	53
Smart-Det (proposed)	75.4%	1.000	0

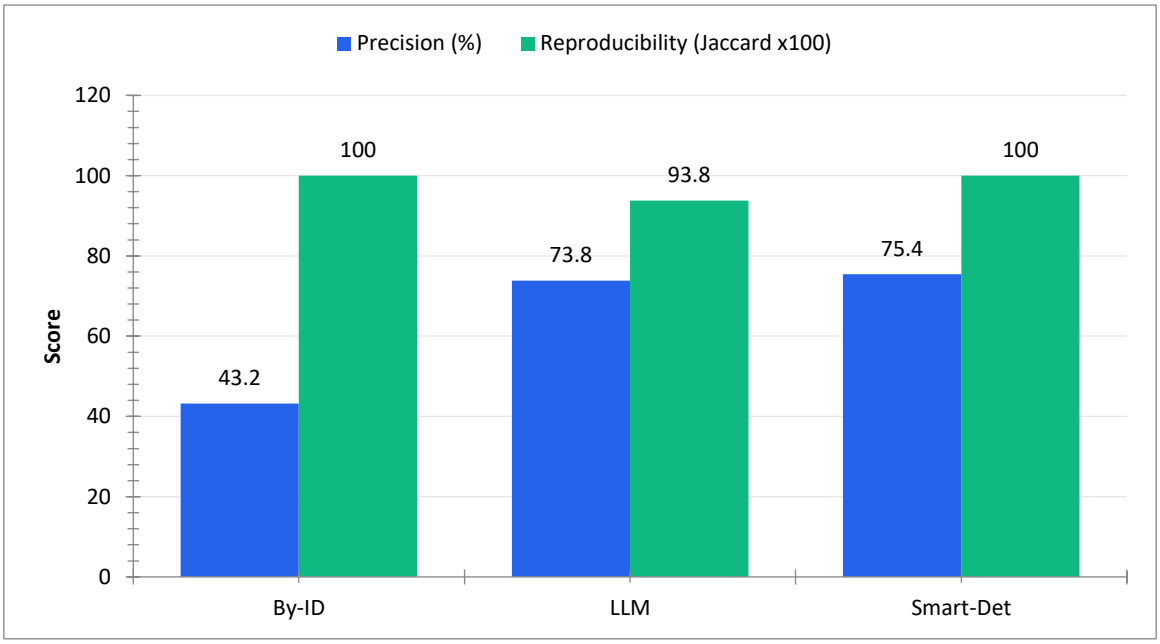


Figure 4-5 Ranking precision and reproducibility (Jaccard × 100) per strategy.

Smart-Det attains the highest precision (75.4%), slightly above the LLM (73.8%), with perfect reproducibility (Jaccard = 1.000) and zero LLM calls. The LLM, by contrast, is only 0.938 reproducible: under identical inputs and temperature 0 it still changed roughly 6% of its selections between runs, providing direct evidence for the claim that LLM ranking is non-deterministic. Replacing the LLM with Smart-Det

therefore preserves indeed slightly improves precision while making the stage fully reproducible and free.

The By-ID baseline is the decisive control. Sorting deterministically by course ID collapses precision to 43.2%, even though it is just as reproducible (Jaccard = 1.000). This shows that the quality does not come from determinism per se but from the relevance scoring (title, coverage, specificity); determinism is what guarantees reproducibility, while the scoring is what guarantees quality. The 6-task reproducibility sample is small and is noted as a limitation; a larger determinism sample is suggested for the final run.

4.7 Discussion

4.7.1 Synthesis: Where the LLM Belongs

Read together, the five experiments form a single architectural argument. P1 shows the LLM is genuinely needed at extraction (F1 96.7% vs. 84.4%, with unique disjunction and level abilities). P4 and P5 show that at search and ranking the LLM can be removed with no loss of quality deterministic pruning is more accurate and far faster (P4), and deterministic ranking matches or beats LLM precision while being perfectly reproducible (P5). P3 shows the deterministic guardrail E(c) is redundant once upstream filtering is in place, and P2 shows that the measurable efficiency gain comes from the deterministic filter rather than the tree. The consistent conclusion is that the LLM should be confined to the one task that requires natural-language understanding, and the remainder of the pipeline should be deterministic; doing so yields a system whose non-extraction stages are reproducible and incur no per-query LLM cost.

4.7.2 Threats to Validity

- The CV–JD test set is synthetic; results may differ on noisier real-world documents.
- The ground truth was produced by a single author with AI-assisted analysis and a human final decision; a second annotator and a Cohen’s kappa agreement score remain desirable.

- Recall in P3 is estimated over a labelled sample (recommended items plus a random pool sample), not the entire catalogue, and therefore over-counts neither relevant nor irrelevant items beyond that sample.
- The reproducibility measurement in P5 uses a 6-task sample to control LLM cost.

4.7.3 Observed Limitations

Two limitations surfaced directly from the experiments and motivate the discussion in Chapter 5. First, recommendation quality is upper-bounded by the quality of course skill tags: a course tagged with a broad skill set can enter a learning path outside its true specialism, a weakness inherent to tag-based matching (for example, an alias that folds a platform name into a broader concept can inject an out-of-context skill). Second, some less common technologies have few dedicated courses in the corpus, which caps achievable coverage regardless of the retrieval method.

CHAPTER 5 CONCLUSION & FUTURE WORK

5.1 Summary of Contributions

This thesis set out to analyse the gap between a candidate and a target job and to recommend courses that close it, while avoiding the two traps that limit existing approaches: the semantic blindness of keyword matching and the unreliability of an end-to-end language model. The work makes four main contributions.

A deterministic, prompt-constrained extraction engine. A prompt-engineered LLM (Qwen3-Coder-480B, temperature 0) turns unstructured CVs and JDs into a structured set of skills with proficiency levels. On 36 hand-labelled job descriptions it reached an F1 of 96.7%, against 84.4% for a keyword baseline, and it was the only method that could read disjunctive "or" clauses (10 of 10) and assign proficiency levels (83.2% correct). This is the one place the system relies on a model, and the experiment shows the reliance is justified.

A quantitative, progressive skill-gap model. A dedicated module computes, for each required skill, both whether it is missing and how many proficiency levels separate the candidate from the target, and it resolves disjunctive and nested requirements without creating phantom gaps. The result is a level-progressive target that orders foundational material before advanced material, which gives the later stages a precise objective to work against.

Deterministic tree-based retrieval and ranking. The system replaces model-driven branch routing with deterministic skill-set matching, and model-driven ranking with a relevance score built from title match, gap coverage, and specificity. The effect was measured rather than assumed. Deterministic pruning recovered 99.2% of the relevant courses where an LLM-routed baseline reached only 63.3%, ran about twenty-five times faster, and made no model calls; deterministic ranking matched the precision of LLM ranking (75.4% against 73.8%) and returned the same courses on every run. End to end, 74.8% of the recommended courses were judged relevant against human labels.

An empirical account of where the language model belongs. Beyond the system itself, the five experiments form a small study of the hybrid design. Their consistent message is that the model can be removed from search and ranking with no measurable loss of quality, while those stages become reproducible and free of per-query cost. Two findings are worth stating plainly because they correct a common intuition: the 99.8% reduction in tokens sent to the model comes from the deterministic filter rather than from the tree, and the guardrail E(c) turns out to be redundant once upstream filtering is in place, since every surviving course already clears its threshold.

5.2 System Limitations

The current system also has clear limits, several of which surfaced directly from the experiments.

5.2.1 Single-label Taxonomy and Tag-based Matching

Each course sits in a single subcategory, so a cross-disciplinary course, such as a full-stack course that covers both frontend and backend, may be pruned from a branch where it would have been useful. More broadly, the quality of the recommendations is bounded by the quality of the course skill tags. A course tagged with a wide set of skills can enter a learning path outside its real specialism, and a small number of aliases that fold a tool into a broader concept can inject a skill that does not match the context. This is an inherent weakness of matching on tags rather than on course content.

5.2.2 Static Data Obsolescence

The catalogue is scraped at a single point in time. EdTech platforms regularly update, retire, or unlist courses, so without a synchronisation mechanism some recommended links may eventually lead to pages that no longer exist.

5.2.3 Cloud API Dependency

Extraction depends on a cloud-hosted model, which adds network latency and token cost and is a single point of failure under heavy load. It is worth noting that this dependency is now confined to the extraction step; retrieval and ranking run locally

and for free, so the exposure is smaller than in an end-to-end LLM design, but it has not been removed.

5.2.4 Extreme Cold-Start Threshold:

The system works from a measurable difference between a CV and a JD. When a candidate has almost no relevant skills, the gap becomes very large, and the learning path can turn exhaustive rather than focused, which is less useful to the learner.

5.2.5 Evaluation Scope and Ground Truth

Finally, the evaluation has its own boundaries. The CV–JD test set is synthetic, so results may differ on noisier real documents. The ground truth was labelled by a single author with AI-assisted analysis and a human final decision; a second independent annotator and a Cohen's kappa agreement score remain desirable. Recall is estimated over a labelled sample rather than the full catalogue, and the ranking-reproducibility measurement used a small six-task sample. These choices were made for feasibility, and they bound how far the reported numbers should be generalised.

5.3 Future Work

Several extensions would address these limits and move the prototype toward a production-ready tool.

5.3.1 From Tree to Knowledge Graph

Replacing the hierarchical tree with a knowledge graph, for example in Neo4j, would let a single course connect to several domains at once. That removes the single-label constraint and allows the search to follow relationships across domains rather than down one fixed branch, which directly addresses the cross-disciplinary case in Section 5.2.1.

5.3.2 Automated Catalogue Maintenance

A scheduled maintenance pipeline would keep the catalogue current. Background jobs could check course URLs and remove dead links, ingest newly published courses, and refresh existing records, so that the educational supply stays accurate over time.

5.3.3 Local LLM Deployment

Moving extraction to a locally hosted open-weight model would remove the API cost and keep candidate CVs on the user's own infrastructure, which matters for privacy. This assumes hardware capable of running the model with acceptable latency, and it would trade some extraction quality for independence from the cloud.

5.3.4 Dynamic Alias Discovery

The alias map is maintained by hand today. A background worker that mines developer forums and code repositories for emerging synonyms and abbreviations could keep the map current with less manual effort, which would also reduce the tag and alias problems noted in Section 5.2.1.

REFERENCES

[1] W. Zhang *et al.*, “LLMTreeRec: Unleashing the power of large language models for cold-start recommendations,” in *Proc. 31st Int. Conf. Computational Linguistics (COLING)*, 2025.

[2] Qwen Team, “Qwen technical report,” arXiv:2309.16609, 2023. [Online]. Available: https://arxiv.org/abs/2309.16609

[3] S. Ramírez, “FastAPI: High performance, easy to learn, fast to code, ready for production,” 2024. Accessed: Jun. 2026. [Online]. Available: https://fastapi.tiangolo.com/

[4] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.

[5] F. Ricci, L. Rokach, and B. Shapira, Eds., *Recommender Systems Handbook*, 3rd ed. New York, NY, USA: Springer, 2022.

[6] Meta Open Source, “React: A JavaScript library for building user interfaces,” 2024. Accessed: Jun. 2026. [Online]. Available: https://react.dev/